Frontend Theory Assignment – 3

1. What is the difference between frontend and backend JavaScript?

JavaScript can be used on both the frontend and backend of a web application.

Frontend JavaScript runs in the user's web browser. It is mainly used to make web pages interactive and dynamic. It can manipulate the DOM, handle user events, validate forms, and change webpage content without reloading the page.

Backend JavaScript runs on the server using environments such as Node.js. It is used to handle server-side operations such as processing requests, communicating with databases, authentication, and creating APIs.

Example:
Frontend JavaScript can validate a login form, while backend JavaScript can verify the login details stored in a database.

In short: Frontend JavaScript works mainly with the user interface, while backend JavaScript handles server-side logic and data processing.

2. What is DOM?

DOM stands for Document Object Model. It is a programming interface that represents an HTML document as a tree of objects or nodes.

Using JavaScript, we can access and modify these nodes. We can change HTML content, attributes, styles, and elements dynamically.

Example:

document.getElementById("heading").innerHTML = "Hello";

Here, JavaScript accesses the HTML element having the ID `heading` and changes its content.

Therefore, the DOM allows JavaScript to dynamically interact with and modify a webpage.

3. What is an event in JavaScript?

An event in JavaScript is an action or occurrence that happens in a webpage and can be detected and handled by JavaScript.

Examples of events include:

- Clicking a button
- Moving the mouse
- Pressing a keyboard key
- Submitting a form
- Loading a webpage

JavaScript uses event handlers or event listeners to perform an action when an event occurs.

Example:

```
button.addEventListener("click", function() {
   alert("Button clicked");
});
```

In this example, the `click` is the event, and the function is executed when the button is clicked.

4. What is an event loop in JavaScript?

The event loop is a mechanism in JavaScript that allows it to handle asynchronous operations even though JavaScript is primarily single-threaded.

When an asynchronous operation such as a timer or network request is performed, JavaScript does not stop executing the rest of the program. After the asynchronous operation is completed, its callback is placed in a queue.

The event loop continuously checks whether the call stack is empty. If it is empty, the event loop moves the waiting callback to the call stack for execution.

Example:

```
console.log("Start");

setTimeout(function() {
   console.log("Hello");
}, 1000);

console.log("End");
```

Output:

```
Start
End
Hello
```

Thus, the event loop helps JavaScript perform non-blocking asynchronous operations.

5. What is hoisting in JavaScript?

Hoisting is a JavaScript behavior in which declarations of variables and functions are processed before the code is executed.

Function declarations can be called before they appear in the program.

Example:

```
hello();

function hello() {
   console.log("Hello");
}
```

This works because the function declaration is hoisted.

For variables declared using `var`, the declaration is hoisted but its value is initially `undefined`.

```
console.log(x);
var x = 10;
```

The output is:

undefined

Variables declared using `let` and `const` are also hoisted internally, but they cannot be accessed before their declaration because they remain in the Temporal Dead Zone (TDZ).

6. What is event bubbling?

Event bubbling is a mechanism in JavaScript in which an event starts from the element where it occurred and then propagates upward through its parent elements.

For example, if a button is inside a `<div>` and the button is clicked, the click event can first occur on the button and then bubble to the `<div>` and other parent elements.

Example:

```
div.addEventListener("click", function() {
   console.log("Div clicked");
});

button.addEventListener("click", function() {
```

```
    console.log("Button clicked");
});
```

When the button is clicked, the button's event handler executes first, followed by the parent's event handler.

Event bubbling is useful for techniques such as event delegation.

7. What is the difference between named function, anonymous function and arrow function?

A named function has a specific name and can be called using that name.

```
function add(a, b) {
    return a + b;
}
```

An anonymous function does not have a name. It is commonly assigned to a variable or passed as a callback.

```
let add = function(a, b) {
    return a + b;
};
```

An arrow function is a shorter way of writing a function using the => syntax.

```
let add = (a, b) => a + b;
```

Difference:

Function Type	Description
Named Function	Has a function name
Anonymous Function	Does not have its own name
Arrow Function	Shorter syntax using =>

Arrow functions also handle the `this` keyword differently from regular functions.

8. Explain for-of and for-in loop.

The for-of loop is used to iterate over the values of an iterable such as an array or string.

Example:

```
let arr = [10, 20, 30];

for (let value of arr) {
    console.log(value);
}
```

Output:

```
10
20
30
```

The for-in loop is used to iterate over the keys or property names of an object. It can also be used with arrays to access their indexes.

Example:

```
let student = {
    name: "John",
    age: 20
};

for (let key in student) {
    console.log(key);
}
```

Output:

```
name
age
```

Difference:

for-of	for-in
Iterates over values	Iterates over keys/properties
Commonly used with arrays and strings	Commonly used with objects
Example: `10, 20, 30`	Example: `0, 1, 2` for array indexes

9. Explain any five array methods.

JavaScript provides several built-in methods for working with arrays. Five important array methods are:

1. push()
Adds one or more elements to the end of an array.

```
let arr = [1, 2];
arr.push(3);
```

Result:

[1, 2, 3]

2. pop()
Removes the last element from an array.

```
let arr = [1, 2, 3];
arr.pop();
```

Result:

[1, 2]

3. shift()
Removes the first element from an array.

```
let arr = [1, 2, 3];
arr.shift();
```

Result:

[2, 3]

4. unshift()
Adds one or more elements to the beginning of an array.

```
let arr = [2, 3];
arr.unshift(1);
```

Result:

[1, 2, 3]

5. slice()
Returns a portion of an array without changing the original array.

```
let arr = [10, 20, 30, 40];
let result = arr.slice(1, 3);
```

Result:

[20, 30]

Thus, array methods make it easier to add, remove, and extract elements from arrays.

10. What is a callback method?

A callback function is a function that is passed as an argument to another function and is executed later by that function.

Callbacks are commonly used for handling events and asynchronous operations.

Example:

```
function greet(name, callback) {
    console.log("Hello " + name);
    callback();
}

function message() {
    console.log("Welcome!");
}

greet("John", message);
```

Output:

```
Hello John
Welcome!
```

Here, `message` is passed as an argument to the `greet` function. It is then called inside `greet`, so `message` is the callback function.

Callbacks are useful when we want one function to execute after another operation is completed.